

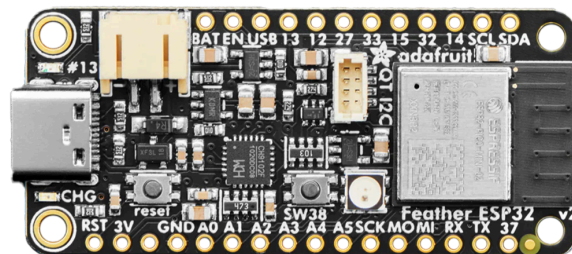
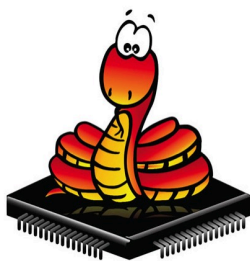
Artificial Intelligence of Things - Intelligent and Connected Systems

EECS E4764
Columbia University

Prof. Xiaofan Jiang

Fall 2026

Lab 3 - Bus Communication: I2C



Introduction

A bus is an interface that allows different components to communicate with each other, so that each component can be modularized and focus on a single task or duty within the larger scope of the system. For example, a modern personal computer generally contains a CPU for processing, memory (usually one of the variants of RAM) for storing programs and instructions that the CPU runs, a hard drive for storing files and user data, a GPU with an LCD display and keyboard/mouse peripherals for user operability, and a sound card. Computers may have less or they may have more components (e.g. laptops generally have WiFi cards for wireless connectivity to the Internet), but all components within a computer communicate with each other through a type of bus that makes it possible for all of the components to function together to allow your own computers to function.

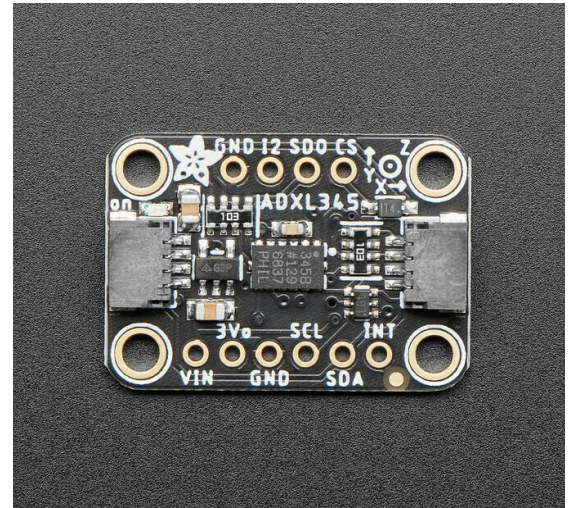
In this and next lab, we will explore two bus protocols that are common and popular in embedded and IoT systems: SPI and I2C. We will use SPI to communicate with an accelerometer and I2C to communicate with an LCD screen and start building our smart watch. To begin building our smartwatch application we must complete the following on our ESP32.

We will also work on implementing time-keeping and alarm functionalities for our smart watch.

Part 1: Getting ready

✓ Task 1

Solder the ADXL345 accelerometer using the same method as Lab2. (It won't be needed in Lab 3, so take your time.)



Part 2: Displaying and Keeping Time

Task 1

Obtain and modify the system time using real-time clock (RTC). Reset the system time to a hardcoded value when the ESP32 boots up.

Task 2

Interface with the LCD screen through I2C and display the system time on the screen; the display should continuously update the time just like in a normal watch. (Remember to copy the [ssd1306.py](#) code to your ESP32.)

Task 3

The OLED display shield comes with 3 buttons. Use these buttons to add functionalities to allow you to change the time and update the LCD display to reflect these changes. Make sure you integrate the debouncing code from Lab 2 to debounce these buttons.

Checkpoint 1

Display the time on the OLED display; the time should update like a regular watch. Be able to change time through the OLED display buttons.

Task 4

Add in the functionality to view values from the light sensor. Additionally, adjust the display brightness or contrast depending on the brightness in the room (the display should get dimmer if the room is dimly lit and brighter if it is bright out).

Checkpoint 2

Adjust screen brightness based on ambient light.

Part 3: Adding an Alarm

Task 1

Add in an alarm functionality that allows you to set an alarm. When the system time reaches this time, the vibration motor/piezzo should start vibrating to indicate that the alarm has gone off. Additionally, there should also be a visual indication.

Checkpoint 3

Add alarm functionality; Be able to set the alarm through the OLED display and when the alarm goes off, a visual and audio notification should go off.

Task 2

Combine the time and alarm functionalities into one system. In other words, combine parts 2 and 3 into one system.

Checkpoint 4

Combine 1, 2, and 3.

Lab 3 Checkpoints:

1. Display the time on the OLED display; the time should update like a regular watch. Be able to change time through the OLED display buttons.
2. Adjust screen brightness based on ambient light.
3. Add alarm functionality; Be able to set the alarm through the OLED display and when the alarm goes off, a visual and audio notification should go off.
4. Combine 1, 2, and 3.

Submission:

Submit the file for each checkpoint onto courseworks → Assignments → Lab3

The submitted files should be named like

```
lab{x}_{member1_uni}_{member2_uni}_{member3_uni}_{member4_uni}_check{X}.py
```

There should be 4 files for this lab.

e.g.

- lab3_uni1_uni2_uni3_uni4_check1.py
- lab3_uni1_uni2_uni3_uni4_check2.py
- lab3_uni1_uni2_uni3_uni4_check3.py
- lab3_uni1_uni2_uni3_uni4_check4.py